

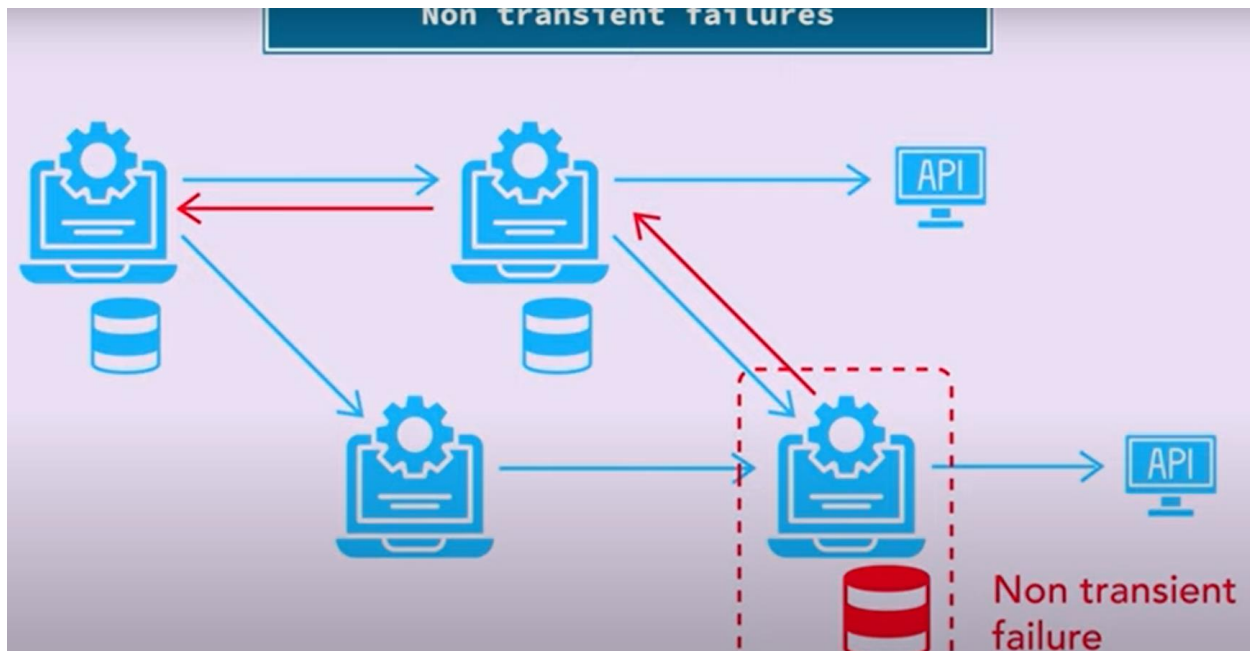
Circuit Breaker Pattern

❖ What is Circuit breaker pattern?

- It is actually a cloud design pattern which is used to deal with non-transient failures with microservices.

❖ What is the meaning of non-transient failure?

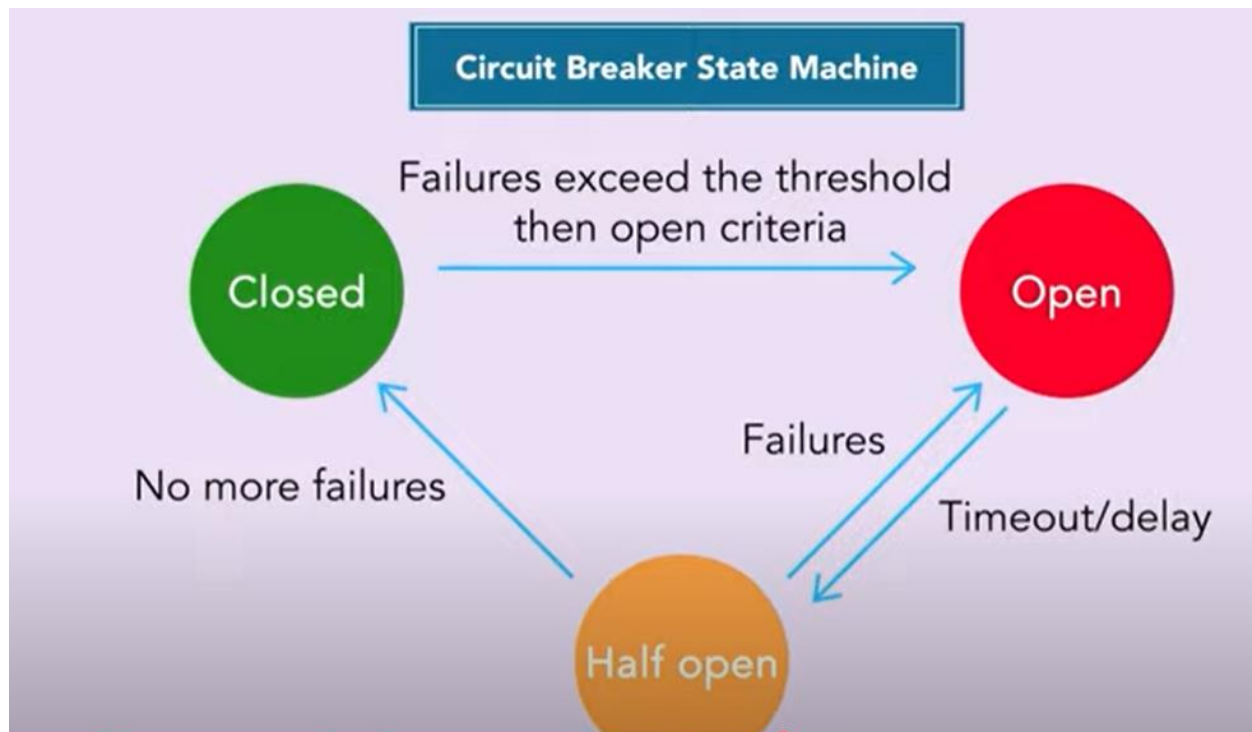
- In the context of the microservices if any failure which can render the system unavailable or the recovery can take longer than few seconds, that failure is constituted as non-transient failure
- Some example of non-transient failures are Time-out or delay where the API takes the longer duration to respond resulting in timeout, or databases going down or connection to the database failing or services bombarded with lot of requests and all the request resulting in failures due to any issue like memory or CPU etc.
- So in the microservices architecture as shown in the below diagram (here DB is failing) so all the microservices dependent on that service will also fail this is called Cascading effects, until this Database will take certain amount of time to recover, so to stop or avoid this cascading effect circuit breaker help while the system recovers.
- In such cases the with the help of circuit breaker we stop the calls to the failing service for some time until the failing system recovers.
- The circuit breaker logic will decide if the requests from service A will be passed through or rejected or returned with an error, so this helps service A not to call service B and so that service B degrades further also service A keep getting error again and again.



❖ How Does Circuit breaker works?

- There are three stages that circuit breakers operates Closed, Half-Open, Open
- By default the circuit breaker will remain in Closed state means the traffic will go from service a to service b as it is and the response will also be returned from service B to service A as it is
- in case the circuit breaker is half open that means some traffic can go for the testing purposes if the calling service can actually call the downstream service and check if there are any failures or not
- in case the circuit breaker is open that means no traffic can go through and the calling service will just be written with that this service is that they are trying to call is not working right now.
- service A calls service B when the circuit breaker is closed the calls from A go to B and the response is written from B as it is
- in case of error scenarios when something goes wrong and service B is degraded the circuit breaker State switches to open State and all the calls that are going to be are rerouted by circuit breaker with a failure and a failure is returned
- In case the circuit breaker is half open that means the service might be recovering or we give it some time to recover and then after some time we try to test if the calls from A to B are going if the calls are successful the response is written to service A as it is and the circuit breaker switches to closed State again

- in the case of half open state if the request is failing or the system has still not recovered then the circuit again goes into the open state
- As Discuss there are these three states closed, open and half open. default state is closed if there are certain failures which exceed threshold then circuit breaker goes into open State stopping any calls from going further being an open State there is a timeout which keeps switching to half open state to check the traffic and to see if the system has recovered if it recovers the half open State goes to close the state if it doesn't then the half open State again goes to open State until the system has recovered, there is a certain timeout or delay which we set in the open State using that timeout the system keeps checking whether the system has recovered or not or the the graded system has recovered from the failure or Not by putting the circuit breaker in half open State here is very simple
- We do not need the circuit breaker pattern if the load is not that heavy or there are very few services.
- We can use circuit breaker if there are lot of services or in distributed system where the multiple services are dependent on each other or we have heavy load, then in that case we can go for circuit breaker pattern implementation.



- In a microservices based ecosystem the services often depends on each other for performing operations and here if one service faces an issue it often creates an cascading failure potentially breaking the entire system, here circuit breaker pattern act

as a safeguard where against such failures preventing them from cascading and even allowing system to operate even when some of the service facing problems.

- Overload protection: it primarily focused on overload protection it aims to prevent service being overwhelms with request when dependent service is slow or unavailable.
- In highly concurrent system failing quickly is often the best strategy. This prevent the single point of failure preventing the chain of failure.
- Circuit breaker pattern preventing cascading failures and provides a way to gracefully handles service failure
- Closed: All calls are allowed, Open state: No calls allowed, Half-Open state: Few Calls allowed (this we can control using Resilience 4J properties)

❖ Circuit Breaker Properties

- slidingWindowType:
 - **Type:** COUNT_BASED or TIME_BASED, **Default:** COUNT_BASED
 - Defines how failures are tracked:
 - **COUNT_BASED:** Based on a fixed number of calls (like 10 calls).
 - **TIME_BASED:** Based on a time window (e.g., last 10 seconds).
 - **For Example:** slidingWindowType: COUNT_BASED
- slidingWindowSize:
 - **Type:** int, **Default:**100
 - The number of calls (if COUNT-based) or time window in seconds (if TIME-based) used to calculate the failure rate. To change the state from CLOSED to OPEN
 - So, if `slidingWindowType = COUNT_BASED` and size is 5 → it evaluates failures among **last 5 calls**.
 - **Example:** slidingWindowSize: 5
- minimumNumberOfCalls:
 - **Default:** 100
 - Minimum number of calls that must be made **before failure rate is calculated**.
 - If fewer calls are made than this, breaker stays in CLOSED
 - Example: minimumNumberOfCalls: 3
- failureRateThreshold:
 - **Default:** 50 (means 50%)

- If more than X% of calls fail in the sliding window, the breaker goes to OPEN
 - So if 3 out of 5 calls fail → breaker trips (60%).
 - **For Example:** failureRateThreshold: 50
- waitDurationInOpenState
 - **Type:** Duration (5s, 1m, etc.), **Default:** 60s
 - How long the breaker should stay **OPEN** before transitioning to **HALF_OPEN** for a test.
 - **For Example:** waitDurationInOpenState: 10s
- permittedNumberOfCallsInHalfOpenState
 - **Default:** 10
 - When breaker is HALF_OPEN, it allows this many test calls:
 - If they succeed → go back to CLOSED
 - If they fail → go back to OPEN
 - For Example: permittedNumberOfCallsInHalfOpenState: 2
- automaticTransitionFromOpenToHalfOpenEnabled
 - **Type:** Boolean, **Default:** false
 - If true, breaker automatically moves from OPEN → HALF_OPEN after waitDurationInOpenState.
 - If false, you have to call an endpoint or logic to trigger the state transition.
 - Always set this to true for automatic recovery
 - For Example: automaticTransitionFromOpenToHalfOpenEnabled: true
- ignoreExceptions:
 - **Type:** List of exception classes
 - Defines exceptions that should **not be treated as failure** (e.g., client errors).
 - For Example:
 - ignoreExceptions:
 - - com.example.exception.BadRequestException
- recordFailurePredicate
 - Advanced Custom Logic
 - Supply a bean that returns true or false for any Throwable
 - This gives **complete control** over failure detection
 - For Example: recordFailurePredicate: com.example.MyFailurePredicate
- registerHealthIndicator:
 - **Type:** Boolean, **Default:** false
 - If true, Spring Boot Actuator will expose circuit breaker health at
 - GET /actuator/health
 - GET /actuator/circuitbreakers
 - For Example: registerHealthIndicator: true
- writableStackTraceEnabled
 - **Default:** true
 - **If false,** avoids filling full stack trace in exceptions for performance

- **allowHealthIndicatorToFail** (Spring Boot 3+ only)
 - **Default:** false
 - If true, circuit breaker's failure state causes health endpoint to return DOWN
- **recordExceptions**
 - **Type:** List of exception classes
 - Defines which exceptions are **counted as failures**
 - For Example:
 - **recordExceptions:**
 - java.io.IOException
 - java.util.concurrent.TimeoutException

For Example:

resilience4j:

circuitbreaker:

instances:

reviewService:

slidingWindowType: COUNT_BASED

slidingWindowSize: 5

minimumNumberOfCalls: 3

failureRateThreshold: 50

waitDurationInOpenState: 10s

permittedNumberOfCallsInHalfOpenState: 2

automaticTransitionFromOpenToHalfOpenEnabled: true

recordExceptions:

- java.io.IOException

- java.util.concurrent.TimeoutException

ignoreExceptions:

- com.example.exceptions.ClientErrorException

registerHealthIndicator: true

writableStackTraceEnabled: false

Summary Table;

Property	Purpose
slidingWindowSize	Number of calls or time to track failures
minimumNumberOfCalls	Minimum requests before triggering logic
failureRateThreshold	% failure to trip the breaker
waitDurationInOpenState	Time breaker stays open
permittedNumberOfCallsInHalfOpenState	Trial calls during half-open
automaticTransitionFromOpenToHalfOpenEnabled	Auto-recover after wait
recordExceptions	Exceptions treated as failure
ignoreExceptions	Exceptions ignored
registerHealthIndicator	Enable actuator health check
writableStackTraceEnabled	Control stack trace performance